

Sprint 15 Structured Notes: Testing Strategy

Topic

Functional, E2E, Usability, Compatibility, Performance, Security, A/B, Alpha/Beta, Regression, and Retesting

0. Where You Are Before Sprint 15

Before Sprint 15, you should already understand these ideas:

- React apps are built from components.
- JSX describes what should appear on the screen.
- Props pass data from parent components to child components.
- State lets React remember changing data.
- Events let React respond to clicks, typing, and form submissions.
- Forms can be tested by checking visible behavior.
- Effects, fetching, routing, Context, and performance basics are already familiar.
- Sprint 14 introduced testing foundations.
- You know the difference between manual testing and automated testing.
- You know that a unit test checks one small piece of code.
- You know basic test syntax such as `test()`, `expect()`, and matchers.
- You know that React Testing Library encourages testing what the user sees and does.

Sprint 14 asked this question:

Can I write a basic test?

Sprint 15 asks a more strategic question:

Which test type should I use for this risk?

That is the main difference.

Sprint 15 is not mainly about writing more `expect()` statements. It is about choosing the correct testing approach.

1. Sprint 15 Goal

The goal of Sprint 15 is to select the right test type for the right risk.

By the end of Sprint 15, you should be able to:

- Explain functional testing.
- Explain non-functional testing.
- Explain smoke testing.
- Use E2E testing for critical user flows.
- Explain usability testing.
- Explain compatibility testing.
- Explain performance testing.
- Explain security testing.
- Explain A/B testing.
- Explain alpha testing.
- Explain beta testing.
- Explain regression testing.
- Explain retesting.
- Match a test type to a realistic project risk.
- Avoid using E2E tests for simple pure functions.
- Create a test strategy for a React app.
- Build a simple QA Plan component.

Simple version:

Sprint 15 teaches you how to stop testing randomly and start testing based on risk.

2. Definition of Done

You are done with Sprint 15 when you can:

- Explain each testing type in simple words.
 - Match each testing type to the risk it protects against.
 - Build the sprint mini-project.
 - Fix the debug task.
 - Answer the revision questions without looking at notes.
 - Explain why E2E should not be used for every tiny detail.
 - Create a basic QA strategy for a React app.
-

3. Scrum Rhythm

Use this rhythm while studying Sprint 15:

Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

What this means:

Step	Meaning
Plan	Decide what you are trying to learn.
Learn	Study the test types.
Build	Create the QA Plan component.
Debug	Fix the broken test strategy labels.
Review	Check whether each test type matches the correct risk.
Refactor	Improve your QA plan so it is clearer.
Retrospective	Reflect on what was confusing and what needs practice.

4. Sprint Backlog

Sprint 15 covers these tasks:

- Differentiate functional and non-functional testing.
 - Explain smoke tests.
 - Explain login scenario testing.
 - Use E2E testing for critical user flows.
 - Explain usability testing types and tools.
 - Explain compatibility testing across browsers, operating systems, hardware, networks, and mobile.
 - Explain load testing, stress testing, soak testing, spike testing, and breakpoint/capacity testing.
 - Explain security testing principles and threats.
 - Explain SAST, DAST, and penetration testing at a beginner level.
 - Explain A/B testing.
 - Explain alpha testing.
 - Explain beta testing.
 - Explain regression testing.
 - Explain retesting.
 - Create a risk-based test plan for a React app.
-

5. Big Mental Model

Different problems need different tests.

A beginner mistake is thinking:

Testing means writing one kind of test for everything.

That is wrong.

A better mental model is:

```
Risk appears
↓
Choose the test type that best catches that risk
↓
Run the test
↓
Use the result to improve confidence
```

Example:

```
Small function may calculate wrong
→ Unit test

Search feature may not filter correctly
→ Functional test

User may not complete checkout
→ E2E test

Users may not understand the layout
→ Usability test

App may break on mobile Safari
→ Compatibility test

App may become slow under heavy traffic
→ Performance test

Private data may be exposed
→ Security test

Two button labels may perform differently
→ A/B test

New release may break old behavior
→ Regression test

A fixed bug may still be broken
→ Retesting
```

Beginner rule:

Do not ask only “Can I test this?” Ask “What risk am I testing?”

6. Test Choice Map

Use this map when choosing a test type:

Situation	Best test type
Small pure logic	Unit test
Feature behavior	Functional test
Basic app health	Smoke test
Critical user journey	E2E test
Confusing design or unclear UX	Usability test
Browser, device, OS, or network risk	Compatibility test
Speed, load, scalability, or responsiveness risk	Performance test
Access, data protection, or attack risk	Security test
A fixed bug needs to be checked again	Retesting
Recent code change may break old behavior	Regression test
Two versions need to be compared with data	A/B test
App needs selected pre-release testers	Alpha testing
App needs limited real-user testing before full launch	Beta testing

Memory rule:

```
Logic problem -> Unit
Feature problem -> Functional
Basic health problem -> Smoke
Journey problem -> E2E
User confusion -> Usability
Device/browser problem -> Compatibility
Speed/load problem -> Performance
Protection problem -> Security
Fixed bug verification -> Retesting
Old feature breaking -> Regression
Version comparison -> A/B
Pre-release selected testers -> Alpha
Pre-release real users -> Beta
```

Part One: Functional and Non-Functional Testing

7. Functional Testing

7.1 What Is Functional Testing?

Functional testing checks whether a feature works as expected.

Simple definition:

Functional testing checks what the app does.

It answers this question:

Does the feature behave correctly?

Examples:

- Can the user log in?
- Does search filter the course list?
- Does clicking Enroll add the course?
- Does the form show an error when the email is invalid?
- Does the cart total update when quantity changes?

7.2 Functional Testing in React

In React, functional testing often checks visible behavior.

Example:

```
Render the Search component.  
Type "React".  
Check that React courses appear.  
Check that unrelated courses disappear.
```

This fits React Testing Library because React Testing Library encourages testing from the user's point of view.

Example shape:

```
render(<Search courses={courses} />);
```

```
await user.type(screen.getByLabelText("Search"), "React");

expect(screen.getByText("React Basics")).toBeInTheDocument();
expect(screen.queryByText("Vue Basics")).not.toBeInTheDocument();
```

This test asks:

When the user searches for React, does the user see React results?

That is a functional test.

7.3 Unit Testing Is Part of Functional Testing

A unit test checks one small piece of behavior.

Example:

```
expect(formatPrice(4)).toBe("$4.00");
```

This is functional because it checks whether the function produces the correct result.

Unit testing is not separate from the idea of functionality. It is a small-scale functional check.

7.4 Smoke Testing Is Also a Basic Functional Check

Smoke testing is a quick basic check before deeper testing.

Example:

```
App opens.
Home page appears.
Login page opens.
Course list appears.
No major crash happens.
```

Smoke testing does not prove the whole app is correct. It only checks whether the app is basically alive.

Beginner rule:

Functional testing checks whether features work.

8. Non-Functional Testing

8.1 What Is Non-Functional Testing?

Non-functional testing checks the quality of the app, not just whether individual features work.

Simple definition:

Non-functional testing checks how well the app behaves.

It answers questions like:

- Is the app fast?
- Is the app usable?
- Is the app secure?
- Is the app reliable?
- Does it work on different browsers?
- Can it handle traffic?
- Does it remain stable over time?

8.2 Functional vs Non-Functional Testing

Type	Main question	Example
Functional testing	Does it work?	Search returns matching courses.
Non-functional testing	How well does it work?	Search remains fast with many courses.

8.3 Beginner Comparison

Functional testing:

The login form lets a valid user log in.

Non-functional testing:

The login form responds quickly and securely under real traffic.

Functional testing checks correctness.

Non-functional testing checks quality.

Beginner rule:

Functional testing checks correctness. Non-functional testing checks quality.

Part Two: Smoke Testing and Login Scenarios

9. Smoke Testing

9.1 What Is a Smoke Test?

A smoke test is a quick basic test that checks whether the main parts of the app are not obviously broken.

Simple definition:

A smoke test checks whether the app is basically working before deeper testing.

The name comes from the idea of turning something on and checking whether smoke appears.

In software, it means checking for obvious failure.

9.2 Smoke Test Example for a React App

Example smoke checklist:

1. Open the app.
2. Check that the Home page loads.
3. Open the Login page.
4. Open the Course List page.
5. Check that the main heading appears.
6. Check that no major crash screen appears.

9.3 What Smoke Testing Is Good For

Smoke testing is good for:

- Catching obvious crashes.
- Checking the app after deployment.
- Checking a build before deeper testing.
- Quickly confirming that the main pages still open.

9.4 What Smoke Testing Is Not Good For

Smoke testing is not enough for:

- Deep business logic.
- Full checkout flows.
- Security validation.

- Performance under load.
- Detailed form behavior.

Bad idea:

The smoke test passed, so the whole app is perfect.

Better idea:

The smoke test passed, so the app is stable enough for deeper testing.

Beginner rule:

Smoke tests are fast safety checks, not complete proof.

10. Login Scenario Testing

Login is a useful example because one feature can need many test types.

A login feature is not just one risk. It has several risks.

10.1 Functional Login Test

Question:

Does login work when the user enters valid details?

BDD-style example:

```
Given the user enters a valid email and password
When the user submits the form
Then the dashboard appears
```

10.2 Validation Login Test

Question:

Does the form reject bad input?

Example:

Given the email field is empty
When the user submits the form
Then an error message appears

10.3 Security Login Test

Question:

Can someone bypass login or access private pages without permission?

Example:

Given the user is logged out
When the user opens /dashboard directly
Then the user is redirected to /login

10.4 E2E Login Flow

Question:

Can the user complete the full login journey in the browser?

Example:

Open app
↓
Go to Login
↓
Type email
↓
Type password
↓
Submit
↓
Dashboard appears

Beginner rule:

One feature can need several test types because it has several risks.

Part Three: E2E Testing and BDD

11. E2E Testing

11.1 What Is E2E Testing?

E2E means End-to-End.

Simple definition:

E2E testing checks whether a real user flow works from start to finish.

It answers this question:

Can a user complete an important journey through the app?

Examples:

- Register -> Log in -> Open dashboard.
- Search course -> Open details -> Enroll.
- Add product to cart -> Checkout -> See confirmation.
- Open donation form -> Fill details -> Submit donation.

11.2 Why E2E Testing Is Useful

E2E testing is useful because it checks many app layers together:

- Routes.
- Forms.
- Buttons.
- Browser behavior.
- API calls.
- Authentication flow.
- Page navigation.
- Real user actions.

11.3 Why E2E Testing Is Expensive

E2E tests can be expensive because they are:

- Slower than unit tests.
- More complex to set up.
- More fragile when UI changes.
- Harder to debug when they fail.
- Dependent on realistic test data.

- Dependent on browser automation.

Beginner rule:

Use E2E tests for critical flows, not every tiny detail.

11.4 Good E2E Test Candidates

Good E2E candidates:

- Signup.
- Login.
- Checkout.
- Course enrollment.
- Payment flow.
- Donation flow.
- Password reset.
- Critical admin action.

These are good E2E candidates because breaking them would seriously harm users or the business.

11.5 Poor E2E Test Candidates

Poor E2E candidates:

- `formatPrice()`.
- `capitalizeName()`.
- `calculateTotal()`.
- `filterCourses()`.
- Small pure helper functions.

Small pure functions should usually be tested with unit tests.

Bad idea:

Use E2E to test `formatPrice()` returns "\$4.00".

Better idea:

Use a unit test for `formatPrice()`.

11.6 E2E Tools

Common E2E and regression testing tools include:

- Playwright.
- Cypress.
- Selenium.
- Puppeteer.

You do not need to master all of them now.

The key idea is that these tools automate browser behavior.

11.7 Playwright-Style E2E Shape

```
import { test, expect } from "@playwright/test";

test("user can open products page", async ({ page }) => {
  await page.goto("/products");

  await expect(
    page.getByRole("heading", { name: "Products" })
  ).toBeVisible();
});
```

Read it like this:

```
Open /products.
Find a heading named Products.
Expect that heading to be visible.
```

11.8 E2E Example for Course Browser

Example flow:

```
User opens the app.
User logs in.
User searches for "React".
User opens a React course.
User clicks Enroll.
User sees "Enrollment successful".
```

Beginner rule:

E2E tests protect the flows that would seriously hurt users or the business if they broke.

12. BDD Scenario Shape

BDD means Behavior-Driven Development.

Sprint 14 introduced BDD. Sprint 15 uses BDD-style scenarios to describe higher-level behavior.

BDD often uses this structure:

```
Given some starting situation
When the user does something
Then the expected result should happen
```

Example:

```
Given the user is logged in
When the user adds a product to cart
Then the cart count should increase
```

Another example:

```
Given the user is on the Course List page
When the user searches for "React"
Then only matching courses should be shown
```

BDD is useful because it describes behavior in plain language before or alongside code.

Beginner rule:

Given/When/Then helps you describe user behavior clearly.

Part Four: Usability Testing

13. Usability Testing

13.1 What Is Usability Testing?

Usability testing checks whether real people can understand and use the app.

Simple definition:

Usability testing checks whether the app is easy and clear for users.

It answers this question:

Can users complete tasks without confusion?

Examples:

- Can users find the search box?
- Do users understand the Enroll button?
- Can users finish checkout without help?
- Do users notice validation errors?
- Does the layout make sense?
- Can users navigate the app easily?

13.2 Usability Testing Is Not Mainly About Code

A React component can be technically correct but still confusing.

Example:

The Enroll button works when clicked.
But users do not notice it.

That is not mainly a code bug. It is a usability problem.

13.3 Usability Testing Types

Type	Simple meaning	Example
Explorative testing	Test early ideas before final design.	Show users a rough course page layout.
Comparative testing	Compare two designs.	Compare two course card layouts.
Assessment testing	Check current usability.	Watch users complete enrollment.

Type	Simple meaning	Example
Validation testing	Confirm final design works.	Verify users can finish the final checkout flow.

13.4 Usability Testing Example

Example usability test:

Ask 5 users to find a React course and enroll.
 Watch where they pause.
 Do not explain the interface while they use it.
 Record what confused them.
 Use the results to improve the UI.

Important point:

In usability testing, do not rescue the user too quickly. If you explain the interface immediately, you lose useful information about what was confusing.

13.5 Usability Testing Tools

Common usability testing tools include:

- Loop11.
- Maze.
- Userbrain.
- UserTesting.
- UXTweak.

You do not need to memorize all tool details now.

Just understand what they help with:

They help observe real users and collect usability feedback.

Beginner rule:

Usability testing checks whether people can use the app, not just whether the code runs.

Part Five: Compatibility Testing

14. Compatibility Testing

14.1 What Is Compatibility Testing?

Compatibility testing checks whether the app works across different environments.

Simple definition:

Compatibility testing checks whether the app works correctly in different browsers, devices, operating systems, networks, and versions.

It answers this question:

Does the app still work outside my own laptop and browser?

14.2 Compatibility Areas

Compatibility testing can include:

Area	Example
Browser	Chrome, Firefox, Safari, Edge
Operating system	Windows, macOS, Android, iOS
Hardware	Low-end phone, high-end laptop, tablet
Network	Fast Wi-Fi, slow mobile network, unstable network
Mobile behavior	Touch input, small screen layout, virtual keyboard
Backward compatibility	Works with older supported versions
Forward compatibility	Still behaves reasonably with newer versions
Input method	Mouse, keyboard, touch
Screen size	Large desktop monitor, tablet, small phone

14.3 Compatibility Testing Example

Example checklist:

Course cards display correctly on:

- Chrome desktop
- Firefox desktop
- Safari mobile
- Chrome Android

- Small screens
- Keyboard navigation

14.4 Why Compatibility Testing Matters

A React app can work perfectly in your browser but break elsewhere.

Examples:

- A layout looks fine on Chrome desktop but breaks on iPhone Safari.
- A hover-only menu works with a mouse but fails on touch screens.
- A form is usable on desktop but hidden behind a mobile keyboard.
- A feature depends on a browser API that is not supported everywhere.

14.5 Beginner Compatibility Strategy

For a beginner React project, test at least:

1. Chrome desktop.
2. Firefox desktop.
3. Safari or mobile Safari if possible.
4. A small mobile screen size.
5. Keyboard navigation.
6. Basic slow-network behavior if the app fetches data.

Beginner rule:

Compatibility testing checks whether the app works in the environments your users actually use.

Part Six: Performance Testing

15. Performance Testing

15.1 What Is Performance Testing?

Performance testing checks whether the app is fast, responsive, scalable, and stable enough.

Simple definition:

Performance testing checks speed, responsiveness, scalability, and stability.

It answers questions like:

- Does the page load quickly?
- Does the UI respond quickly after a click?
- Can the backend handle expected traffic?
- Does the app stay stable after running for a long time?
- What happens when traffic suddenly increases?
- At what point does the system start to fail?

15.2 Performance Testing Is Not Usability Testing

Performance testing and usability testing can both affect user experience, but they are not the same.

Usability question:

Can users understand how to use the app?

Performance question:

Does the app stay fast and stable?

Wrong:

Use usability testing to measure server load under 10,000 users.

Correct:

Use performance/load testing to measure server load under 10,000 users.

15.3 Load Testing

Load testing checks whether the app can handle expected traffic.

Simple definition:

Load testing checks normal or expected high traffic.

Example:

Can the Course Browser handle 1,000 users at the same time?

Use load testing when you want to know whether the app can handle realistic traffic.

15.4 Stress Testing

Stress testing checks what happens under extreme traffic.

Simple definition:

Stress testing pushes the system beyond normal limits.

Example:

What happens if 50,000 users use the Course Browser at the same time?

Stress testing helps you find breaking points and failure behavior.

15.5 Soak Testing

Soak testing checks whether the app stays stable over a long time.

Simple definition:

Soak testing checks long-running stability.

Example:

Can the app run under steady traffic for 24 hours without memory leaks or slowdowns?

Soak testing is useful for finding problems that only appear after time passes.

15.6 Spike Testing

Spike testing checks sudden traffic jumps.

Simple definition:

Spike testing checks what happens when traffic suddenly rises or drops.

Example:

The app usually has 100 users.
Suddenly, 5,000 users arrive after a course launch announcement.
Does the app stay usable?

15.7 Breakpoint or Capacity Testing

Breakpoint testing, also called capacity testing, increases load until the system starts to degrade.

Simple definition:

Breakpoint testing finds the point where the system starts failing or slowing badly.

Example:

Increase traffic gradually:

500 users

1,000 users

2,000 users

5,000 users

10,000 users

Find when response time becomes unacceptable.

15.8 Performance Testing Summary

Type	Main question
Load testing	Can it handle expected traffic?
Stress testing	What happens under extreme traffic?
Soak testing	Does it stay stable over a long time?
Spike testing	What happens after sudden traffic changes?
Breakpoint/capacity testing	At what point does it start to degrade?

Beginner rule:

Performance testing checks whether the app stays fast, responsive, and stable under real or difficult conditions.

Part Seven: Security Testing

16. Security Testing

16.1 What Is Security Testing?

Security testing checks whether the app is protected from misuse, leaks, and attacks.

Simple definition:

Security testing checks whether the app protects users, data, and access correctly.

It answers questions like:

- Can a logged-out user access a private page?
- Can a normal user access admin features?
- Is private data exposed?
- Can bad input break the app?
- Can an attacker inject harmful code?
- Can the app resist common attacks?

16.2 Security Testing Principles

Security testing often checks these ideas:

Principle	Simple meaning	Example
Confidentiality	Private data should stay private.	Other users cannot see my account details.
Integrity	Data should not be changed incorrectly.	A user cannot change another user's course progress.
Authentication	The app knows who the user is.	Login verifies the user identity.
Authorization	The app knows what the user is allowed to do.	Only admins can open the admin page.
Availability	The app stays available when needed.	The app resists traffic problems or attacks.
Non-repudiation	Actions can be traced so users cannot falsely deny them.	Important actions are logged.

16.3 Authentication vs Authorization

These are easy to confuse.

Authentication asks:

Who are you?

Authorization asks:

What are you allowed to do?

Example:

```
Login proves identity.  
That is authentication.
```

```
Checking whether the logged-in user can open /admin checks permission.  
That is authorization.
```

16.4 Common Security Threats

Common threats include:

- XSS.
- SQL injection.
- DoS attacks.
- Broken authentication.
- Broken authorization.
- Exposed private data.

16.5 XSS

XSS means Cross-Site Scripting.

Simple definition:

XSS happens when an attacker gets harmful script code to run in another user's browser.

Example risk:

```
A comment field allows harmful HTML or JavaScript.  
Another user opens the page.  
The harmful script runs in that user's browser.
```

React escapes normal text by default, which helps. But you still need to be careful with dangerous HTML rendering, user-generated content, and backend validation.

16.6 SQL Injection

SQL injection happens when unsafe input changes a database query.

Simple definition:

SQL injection is an attack where user input is used to manipulate a database query.

This is mostly a backend risk, but frontend developers should still understand it.

Bad beginner assumption:

I validated the form in React, so the backend is safe.

Correct thinking:

Frontend validation improves user experience, but backend validation protects the system.

16.7 DoS Attacks

DoS means Denial of Service.

Simple definition:

A DoS attack tries to make the app unavailable by overwhelming it.

Example:

Too many fake requests hit the server.
Real users cannot use the app.

16.8 Frontend Checks Are Not Enough

Important beginner warning:

Frontend checks are not real security by themselves.

Why?

Because users can bypass frontend code.

For example, hiding an Admin button in React is not enough. The backend must still reject unauthorized admin requests.

Correct approach:

Frontend:
Hide or redirect for good user experience.

Backend:
Enforce real access control.

16.9 Security Testing Tool Categories

Security testing tools often fall into these categories:

Tool category	Meaning
SAST	Static Application Security Testing. Checks source code without running the app.
DAST	Dynamic Application Security Testing. Checks the running app from the outside.
Penetration testing	A human or team actively tries to find exploitable weaknesses.

16.10 SAST

SAST checks code before or without running the app.

Simple definition:

SAST scans source code for possible security problems.

Example:

A scanner warns about unsafe use of secrets, risky patterns, or vulnerable code.

16.11 DAST

DAST checks a running app.

Simple definition:

DAST tests the app from the outside while it is running.

Example:

A scanner sends suspicious requests to see whether the app responds unsafely.

16.12 Penetration Testing

Penetration testing is more manual and realistic.

Simple definition:

Penetration testing is when security testers actively try to find real weaknesses.

It is usually more advanced than beginner React testing, but you should know what it means.

Beginner rule:

Security testing checks whether users, data, and permissions are protected.

Part Eight: A/B, Alpha, and Beta Testing

17. A/B Testing

17.1 What Is A/B Testing?

A/B testing compares two versions and uses data to choose the better one.

Simple definition:

A/B testing compares two versions of something to see which performs better.

Example:

Version A: "Start Learning"

Version B: "View Courses"

Metric:

Click-through rate from Home page to Course List.

Decision:

Choose the version with the higher click-through rate after enough data is collected.

17.2 What A/B Testing Answers

A/B testing answers product or design questions such as:

- Which button label gets more clicks?
- Which signup page layout converts better?
- Which pricing page leads to more upgrades?

- Which onboarding flow helps more users finish setup?

17.3 Good A/B Test Candidates

Good A/B candidates:

- CTA button labels.
- Page headlines.
- Signup page layouts.
- Pricing page designs.
- Onboarding flows.

17.4 What A/B Testing Is Not For

A/B testing is not for checking security bugs or code correctness.

Bad:

Use A/B testing to find SQL injection issues.

Good:

Use security testing to find SQL injection issues.

Bad:

Use A/B testing to test whether `formatPrice()` works.

Good:

Use a unit test to test `formatPrice()`.

17.5 A/B Testing Tools

Common A/B or feature-flag experimentation tools include:

- GrowthBook.
- LaunchDarkly.

17.6 A/B Test Plan Example

Experiment:
Compare two CTA button labels.

Version A:
"Start Learning"

Version B:
"View Courses"

Success metric:
Click-through rate from home page to course list.

Decision rule:
Choose the version with the higher click-through rate after enough data is collected.

Beginner rule:

A/B testing compares product or design choices using data.

18. Alpha Testing

18.1 What Is Alpha Testing?

Alpha testing happens before public release with selected testers.

Simple definition:

Alpha testing is early pre-release testing with a controlled group.

Example:

Your team or a small invited group tests the Course Browser before real users get it.

18.2 Who Does Alpha Testing?

Alpha testers might be:

- Developers.
- QA testers.

- Internal team members.
- Selected trusted users.
- Stakeholders.

18.3 What Alpha Testing Is Good For

Alpha testing helps catch:

- Major bugs.
- Missing features.
- Confusing flows.
- Broken pages.
- Early usability problems.
- Release-blocking issues.

Beginner rule:

Alpha testing happens before broad real-user release.

19. Beta Testing

19.1 What Is Beta Testing?

Beta testing happens with real users before the full public release.

Simple definition:

Beta testing is limited real-user testing before full launch.

Example:

You release the Course Browser to 50 real learners before launching it to everyone.

19.2 Alpha vs Beta Testing

Type	Who tests?	When?	Main purpose
Alpha testing	Selected internal or controlled testers	Earlier before release	Catch major issues early.
Beta testing	Real users	Later before full release	Learn how the app behaves with real users.

Short memory rule:

Alpha = early controlled testers
Beta = limited real users

19.3 What Beta Testing Is Good For

Beta testing helps find:

- Real-world bugs.
- Device-specific problems.
- Confusing user flows.
- Missing expectations.
- Performance issues under real usage.
- Feedback before full launch.

Beginner rule:

Beta testing gives you real-world feedback before everyone gets the app.

Part Nine: Regression Testing and Retesting

20. Regression Testing

20.1 What Is Regression Testing?

Regression testing checks that old behavior still works after changes.

Simple definition:

Regression testing checks that new changes did not accidentally break existing features.

It answers this question:

Did something that used to work break again?

20.2 Why Regression Testing Matters

When you change code, you can accidentally break unrelated features.

Examples:

- You change login and accidentally break protected routes.
- You change course cards and accidentally break course search.

- You update the API data shape and accidentally break the dashboard.
- You refactor checkout and accidentally break payment confirmation.

Regression testing protects old behavior while you add new behavior.

20.3 Regression Testing Example

Change made:

The course card layout was redesigned.

Regression tests to rerun:

Course list renders.
Search still filters courses.
Course detail links still work.
Enroll button still works.
Checkout flow still works.

20.4 Regression Testing Tools

E2E tools can also be used for regression tests, especially for repeated browser flows.

Common tools include:

- Playwright.
- Cypress.
- Selenium.
- Puppeteer.

Beginner rule:

Regression testing protects existing behavior after code changes.

21. Retesting

21.1 What Is Retesting?

Retesting checks whether a specific fixed bug is actually fixed.

Simple definition:

Retesting means testing the same failed case again after a fix.

Example:

Bug:
Login failed when email had uppercase letters.

Fix:
The login normalization logic was updated.

Retest:
Try the same uppercase email login case again.

21.2 Retesting vs Regression Testing

These two are easy to confuse.

Situation	Correct test type
Checking whether one known bug was fixed	Retesting
Checking whether a new change broke old features	Regression testing

21.3 Simple Difference

Retesting asks:

Is this specific fixed bug really fixed?

Regression testing asks:

Did this change break anything else that used to work?

21.4 Example Difference

Bug:

Enroll button did not work on mobile.

After fixing it:

Retesting:
Try the Enroll button on mobile again.

Regression testing:

Also check course search, course detail, login, and enrollment success message to make sure the fix did not break other things.

Beginner rule:

Retesting checks the known fix. Regression testing checks for unexpected breakage.

Part Ten: Course Browser Test Strategy

22. Example Project: Course Browser

Assume you are building a Course Browser app.

The app has:

- Home page.
- Login page.
- Course list page.
- Course detail page.
- Search input.
- Enroll button.
- Dashboard.
- Protected routes.
- API data.

Sprint 15 asks you to create a testing strategy for this type of app.

23. Unit Tests for Course Browser

Use unit tests for small pure logic.

Good unit test targets:

```
- formatPrice()  
- calculateCourseProgress()  
- filterCourses()  
- capitalizeCourseTitle()  
- getCompletedLessonCount()
```

Example:

```
test("formats price", () => {  
  expect(formatPrice(4)).toBe("$4.00");  
});
```

Why unit test?

Because `formatPrice()` is small pure logic. It does not need a browser journey.

24. Functional Tests for Course Browser

Use functional tests for feature behavior.

Functional test examples:

- Search filters the course list.
- Empty search shows all courses.
- Login form shows validation errors.
- Enroll button changes the UI after click.
- Course detail page shows the selected course.
- Protected page redirects logged-out users.

Example BDD shape:

```
Given the Course List page has several courses  
When the user searches for "React"  
Then only matching React courses are shown
```

25. E2E Flow for Course Browser

Use E2E for one critical journey.

Example:

```
User opens the app.  
User logs in.  
User searches for a course.  
User opens course detail.
```

User clicks Enroll.
User sees a success message.

Why E2E?

Because this checks a full user journey across routes, forms, UI, and app behavior.

26. Compatibility Checks for Course Browser

Compatibility checklist:

- Chrome desktop.
- Firefox desktop.
- Safari mobile.
- Chrome Android.
- Small screen layout.
- Keyboard navigation.
- Touch interaction.
- Slow mobile network behavior.

Why compatibility testing?

Because course cards and navigation may work on desktop but fail on mobile or Safari.

27. Performance Checks for Course Browser

Performance checklist:

- Course search stays responsive with many courses.
- Course list does not freeze the page.
- API loading state appears quickly.
- Dashboard does not become slow after many items.
- App handles expected user traffic.

Possible test type:

Load testing:
Can the app handle expected traffic?

Stress testing:

What happens under extreme traffic?

Spike testing:

What happens after sudden launch-day traffic?

28. Security Checks for Course Browser

Security checklist:

- Logged-out users cannot open /dashboard directly.
- Normal users cannot open admin pages.
- Backend rejects unauthorized requests.
- User input is handled safely.
- Private user data is not exposed.

Important:

React route protection improves user experience, but backend authorization must enforce real security.

29. Regression Triggers for Course Browser

Rerun important tests after:

- Any change to login.
- Any change to routing.
- Any change to course enrollment.
- Any change to API response shape.
- Any change to course card layout.
- Any change to protected routes.
- Any dependency update that affects routing, forms, or auth.

Why?

Because these areas affect important user flows.

30. Full Course Browser Test Strategy Example

Unit tests:

- formatPrice()
- calculateCourseProgress()
- filterCourses()

Functional tests:

- Search filters course list.
- Empty search shows all courses.
- Login form shows validation errors.
- Enroll button changes UI after click.

E2E flow:

- User logs in.
- Searches for a course.
- Opens course detail.
- Clicks Enroll.
- Sees success message.

Compatibility checks:

- Chrome desktop.
- Firefox desktop.
- Safari mobile.
- Small screen layout.
- Keyboard navigation.

Performance checks:

- Search remains responsive with a large course list.
- App handles expected traffic.
- Loading states appear during slow API calls.

Security checks:

- Logged-out users cannot access /dashboard directly.
- Normal users cannot access admin-only pages.
- Backend rejects unauthorized requests.

Regression triggers:

- Any change to login.
- Any change to routing.
- Any change to course enrollment.
- Any change to API response shape.

Part Eleven: QA Plan Component

31. Mini Project: QA Plan Component

The Sprint 15 mini-project is a one-page QA Plan component.

The component should:

- Show test categories.
- Show the risk each category catches.
- Show one example for each category.
- Render the categories from an array.
- Use a stable key.

This practices:

- React list rendering with `map()`.
- Stable keys.
- Structuring data.
- Explaining test types.
- Matching risks to test categories.
- Thinking like a QA planner.

Beginner rule:

A QA plan does not only say “write tests.” It says which risks each test protects.

32. QA Plan Component Code

```
const qaCategories = [
  {
    type: "Unit Test",
    risk: "Small logic may calculate the wrong result",
    example: 'formatPrice(4) should return "$4.00"',
  },
  {
    type: "Functional Test",
    risk: "A feature may not behave correctly",
    example: "Search should show matching courses",
  },
  {
    type: "Smoke Test",
    risk: "The app may be obviously broken after deployment",
  },
]
```

```

    example: "Home, Login, and Course List pages open without crashing",
  },
  {
    type: "E2E Test",
    risk: "A critical user journey may break",
    example: "User logs in, opens a course, enrolls, and sees success",
  },
  {
    type: "Usability Test",
    risk: "Users may not understand how to use the app",
    example: "Watch users try to find and enroll in a React course",
  },
  {
    type: "Compatibility Test",
    risk: "The app may break on some browsers or devices",
    example: "Course cards work on iPhone Safari and Chrome Android",
  },
  {
    type: "Performance Test",
    risk: "The app may become slow under traffic or large data",
    example: "Search stays responsive with 10,000 courses",
  },
  {
    type: "Security Test",
    risk: "Private data or protected routes may be exposed",
    example: "Logged-out users cannot open /dashboard directly",
  },
  {
    type: "A/B Test",
    risk: "We may not know which design performs better",
    example: 'Compare "Start Learning" vs "View Courses"',
  },
  {
    type: "Alpha Test",
    risk: "Major issues may exist before public release",
    example: "Selected testers try the app before release",
  },
  {
    type: "Beta Test",
    risk: "Real users may find issues before full launch",
    example: "A small group of real learners uses the app before public launch",
  },
  {
    type: "Regression Test",
    risk: "New changes may break old behavior",
    example: "After changing course cards, rerun search and enroll tests",
  },
  {

```



```

    type: "Retesting",
    risk: "A known bug fix may not actually work",
    example: "After fixing login, test the same failed login case again",
  },
];

export default function QAPlan() {
  return (
    <main>
      <h1>QA Plan for Course Browser</h1>
      <p>
        This plan matches each test type to the risk it is designed to catch.
      </p>

      <section>
        {qaCategories.map((item) => (
          <article key={item.type}>
            <h2>{item.type}</h2>
            <p>
              <strong>Risk:</strong> {item.risk}
            </p>
            <p>
              <strong>Example:</strong> {item.example}
            </p>
          </article>
        ))}
      </section>
    </main>
  );
}

```

33. What This Component Practices

This component practices:

- Rendering a list with `map()`.
- Using a stable key.
- Keeping repeated UI data in an array.
- Showing simple structured information.
- Matching test types to risks.
- Explaining testing strategy clearly.

Important React note:

```
key={item.type}
```

This is acceptable here because each test type is unique and stable.

Do not use `Math.random()` as a key.

Part Twelve: Practice Block

34. Mandatory Practice Coding Block

34.1 Warm-Up Drill 1: Write One Smoke Test Idea

Example answer:

```
The app opens, the Home page heading appears, and the Course List page loads.
```

34.2 Warm-Up Drill 2: Write One E2E Scenario

Example answer:

```
User logs in, searches for "React", opens a course, clicks Enroll, and sees a success message.
```

34.3 Warm-Up Drill 3: Write One Compatibility Checklist Item

Example answer:

```
Course cards must display correctly on iPhone Safari and Chrome Android.
```

35. Guided Coding Task

Create a test strategy for your Course Browser.

Include:

- Unit tests.

- Functional tests.
- One E2E flow.
- Compatibility checks.
- Regression triggers.

Example answer:

Unit tests:

- formatPrice()
- calculateCourseProgress()
- filterCourses()

Functional tests:

- Search filters course list.
- Empty search shows all courses.
- Login form shows validation errors.
- Enroll button changes UI after click.

E2E flow:

- User logs in.
- Searches for a course.
- Opens course detail.
- Clicks Enroll.
- Sees success message.

Compatibility checks:

- Chrome desktop.
- Firefox desktop.
- Safari mobile.
- Small screen layout.
- Keyboard navigation.

Regression triggers:

- Any change to login.
- Any change to routing.
- Any change to course enrollment.
- Any change to API response shape.

36. Debug Task

Fix the test type labels in this broken strategy.

Broken strategy:

- Use E2E to test `formatPrice()` returns "\$4.00".
- Use unit testing to test full checkout across browser and payment page.
- Use A/B testing to find SQL injection issues.
- Use usability testing to measure server load under 10,000 users.

Correct answer:

- Use unit testing to test `formatPrice()` returns "\$4.00".
- Use E2E testing to test full checkout across browser and payment page.
- Use security testing to find SQL injection issues.
- Use performance/load testing to measure server load under 10,000 users.

Why:

Problem	Correct test
Small pure function	Unit test
Full browser/payment journey	E2E test
Injection attack	Security test
Heavy traffic/load	Performance/load test

37. Build Something Small

Build a one-page QA Plan component.

Requirements:

- Show test categories.
- Show the risk each category catches.
- Show one example for each category.
- Render the categories from an array.
- Use a stable key.

Minimum categories to include:

Unit Test
Functional Test
Smoke Test
E2E Test
Usability Test
Compatibility Test

```
Performance Test
Security Test
A/B Test
Alpha Test
Beta Test
Regression Test
Retesting
```

38. Challenge Task

Add a fake A/B test plan for a CTA button.

Example:

```
Experiment:
Compare two CTA button labels.

Version A:
"Start Learning"

Version B:
"View Courses"

Success metric:
Click-through rate from home page to course list.

Decision rule:
Choose the version with the higher click-through rate after enough data is
collected.
```

39. Expected Output / Self-Check

You are successful when:

- Every test type is matched to a realistic risk.
- E2E is not overused for simple pure functions.
- Security risks are matched to security testing.
- Load and traffic risks are matched to performance testing.
- A/B testing is used only for comparing versions.
- Regression testing is used after changes.
- Retesting is used for known bug fixes.
- The QA Plan component renders from an array.

- The QA Plan component uses a stable key.
-

Part Thirteen: Revision Checkpoint Questions

40. Checkpoint Question 1

Question:

What is the difference between functional and non-functional testing?

Expected answer:

Functional testing checks whether features work correctly. Non-functional testing checks how well the app behaves, such as performance, reliability, usability, compatibility, and security.

41. Checkpoint Question 2

Question:

Why are E2E tests usually reserved for critical flows?

Expected answer:

Because E2E tests are useful but slower, more complex, and more expensive to maintain than smaller tests. They should protect important user journeys like login, checkout, enrollment, or payment.

42. Checkpoint Question 3

Question:

Name three security testing principles.

Possible answer:

Confidentiality
Integrity
Authentication
Authorization

Availability
Non-repudiation

Any three are acceptable.

43. Checkpoint Question 4

Question:

What is the difference between retesting and regression testing?

Expected answer:

Retesting checks whether a specific fixed bug is fixed. Regression testing checks whether recent changes accidentally broke other existing features.

44. Extra Checkpoint Questions

Answer these without looking:

1. What risk does smoke testing catch?
 2. What is one good E2E test candidate?
 3. What is one poor E2E test candidate?
 4. What is the difference between authentication and authorization?
 5. What is A/B testing used for?
 6. What is alpha testing?
 7. What is beta testing?
 8. What is load testing?
 9. What is stress testing?
 10. What is soak testing?
 11. What is spike testing?
 12. What is breakpoint/capacity testing?
 13. What does SAST mean?
 14. What does DAST mean?
 15. Why is frontend-only security not enough?
-

Part Fourteen: Common Beginner Mistakes

45. Mistake 1: Using E2E for Everything

Bad:

```
Use E2E for formatPrice().  
Use E2E for capitalizeName().  
Use E2E for calculateTotal().
```

Better:

```
Use unit tests for small pure functions.  
Use E2E only for critical user journeys.
```

Rule:

E2E is powerful, but expensive. Do not use it for tiny logic.

46. Mistake 2: Confusing Usability and Performance

Bad:

```
Use usability testing to check server load.
```

Better:

```
Use performance/load testing to check server load.
```

Rule:

Usability checks clarity. Performance checks speed and stability.

47. Mistake 3: Using A/B Testing for Security

Bad:

```
Use A/B testing to find SQL injection issues.
```

Better:

Use security testing to find SQL injection issues.

Rule:

A/B testing compares versions. It does not check attacks.

48. Mistake 4: Thinking Smoke Tests Prove Everything

Bad:

The app opened, so everything works.

Better:

The app opened, so it is ready for deeper testing.

Rule:

Smoke tests catch obvious failure, not every bug.

49. Mistake 5: Confusing Regression Testing and Retesting

Bad:

A bug was fixed, so rerunning every old test is called retesting.

Better:

Testing the fixed bug again is retesting.
Rerunning old tests to catch accidental breakage is regression testing.

Rule:

Retesting checks the known bug. Regression checks old behavior after changes.

50. Mistake 6: Trusting Frontend Security Alone

Bad:

The Admin button is hidden, so users cannot access admin features.

Better:

The frontend can hide the Admin button, but the backend must reject unauthorized admin requests.

Rule:

Frontend checks improve UX. Backend checks enforce security.

Part Fifteen: Sprint Retrospective

51. General Retrospective Prompts

After practicing Sprint 15, answer these:

- What felt easy?
 - What broke first?
 - What concept needs one more tiny app?
 - What would you refactor if this became a real project?
-

52. Sprint 15 Specific Reflection Questions

Answer these:

- Which test type was easiest to understand?
 - Which test type was easiest to confuse with another?
 - Did you overuse E2E testing anywhere?
 - Did you clearly separate security, performance, and usability risks?
 - Can your QA Plan explain why each test type exists?
 - Did you include both regression testing and retesting?
 - Did you include A/B testing only for comparing versions?
 - Did your compatibility testing include browser, mobile, and keyboard checks?
-

Part Sixteen: Sprint 15 Checklist

53. Ready-to-Move-On Checklist

You are ready to move on from Sprint 15 when you can:

- Explain functional testing.
- Explain non-functional testing.
- Explain smoke testing.
- Explain E2E testing.
- Explain why E2E tests are used for critical flows.
- Name common E2E tools such as Playwright, Cypress, Selenium, and Puppeteer.
- Explain BDD-style Given/When/Then scenarios.
- Explain usability testing.
- Name usability testing types: explorative, comparative, assessment, and validation.
- Name usability testing tools such as Loop11, Maze, Userbrain, UserTesting, and UXTweak.
- Explain compatibility testing.
- List compatibility areas: browser, OS, hardware, network, mobile, backward compatibility, and forward compatibility.
- Explain performance testing.
- Explain load testing.
- Explain stress testing.
- Explain soak testing.
- Explain spike testing.
- Explain breakpoint or capacity testing.
- Explain security testing.
- Explain confidentiality, integrity, authentication, authorization, availability, and non-repudiation.
- Recognize threats such as XSS, SQL injection, and DoS.
- Understand SAST, DAST, and penetration testing at a beginner level.
- Explain A/B testing.
- Name A/B testing tools such as GrowthBook and LaunchDarkly.
- Explain alpha testing.
- Explain beta testing.
- Explain regression testing.
- Explain retesting.
- Correctly fix the broken strategy labels.
- Create a Course Browser test strategy.
- Build a QA Plan component.
- Avoid using E2E tests for simple pure functions.

Part Seventeen: One-Page Memory Summary

54. Sprint 15 Core Idea

Sprint 15 is about testing strategy.

The main idea is:

Choose the test type based on the risk.

55. Memory Summary

Functional testing checks whether features work.

Non-functional testing checks quality, such as performance, usability, compatibility, security, and reliability.

Smoke testing checks whether the app is basically alive.

E2E testing checks complete user journeys like login, checkout, enrollment, registration, or donation.

E2E tests are powerful but expensive, so reserve them for critical flows.

Usability testing checks whether real people can understand and use the app.

Compatibility testing checks browsers, operating systems, hardware, networks, mobile behavior, backward compatibility, and forward compatibility.

Performance testing checks speed, responsiveness, scalability, and stability.

Load testing checks expected traffic.

Stress testing checks extreme traffic.

Soak testing checks long-running stability.

Spike testing checks sudden traffic jumps.

Breakpoint or capacity testing finds when the system starts to degrade.

Security testing checks protection: confidentiality, integrity, authentication, authorization, availability, and non-repudiation.

Common security threats include XSS, SQL injection, and DoS.

Security tool categories include SAST, DAST, and penetration testing.

A/B testing compares two versions and uses data, such as click-through rate, to choose the better version.

Alpha testing uses selected testers before release.

Beta testing uses real users before full launch.

Regression testing reruns tests after changes to catch unintended breakage.

Retesting checks whether a known fixed bug is actually fixed.

56. Final Beginner Rule

Do not test randomly.

Think like this:

What can go wrong?

↓

Which risk is that?

↓

Which test type catches that risk best?

That is Sprint 15.